

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Fundada en 1551

FACULTAD DE INGENIERÍA DE SISTEMA E INFORMÁTICA
ESCUELA PROFESIONAL DE INGENIERÍA DE SOFTWARE - LIMA



Proyecto Grupal

Integrantes:

Guevara Chavez Luis Rodrigo

Núñez Cardenas Ivan Joaquin

Bejar Mallma Harian Aaron

Arancivia Salas Christian Gabriel

Rojas Rojas Max Fernando

Docente:

Juan Gamarra Moreno

Asignatura:

Inteligencia Artificial

Documentación de prompts.....	3
Informe 1.....	3
Informe 2.....	7
Informe 3.....	12
Informe 4.....	16
Informe 5.....	19
Informe 6.....	23

Documentación de prompts

Informe 1

N°	P01
Objetivo del prompt	Obtener la definición formal del problema del 8-puzzle y la estructura base del código en Java, incluyendo representación de estados, operadores y verificación de solubilidad.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Genera clases de código para implementar el problema del 8-puzzle en Java. Por favor, proporciona: 1. La definición formal del problema (estados, estado inicial, estado meta, operadores/acciones, costos). 2. La estructura de una clase Estado que represente el tablero 3x3 usando un arreglo unidimensional de 9 posiciones. 3. Métodos para: generar movimientos válidos (ARRIBA, ABAJO, IZQUIERDA, DERECHA), calcular la distancia Manhattan, y verificar si el puzzle es soluble (usando inversiones). 4. Un método main que permita ingresar un estado inicial por teclado y muestre el estado en formato de tablero 3x3. El código debe estar en Java, bien comentado y con buenas prácticas de programación".
Resultado / cómo se usó	Se obtuvo una clase Estado completa con constructor, métodos para movimientos, cálculo de Manhattan y verificación de solubilidad. Se incorporó como base para todos los algoritmos de búsqueda posteriores. La verificación de solubilidad permitió descartar estados no resolubles antes de ejecutar cualquier algoritmo.
Validación / ajuste del equipo	Se verificó que el cálculo de inversiones fuera correcto (incluyendo el espacio vacío). Se ajustó el método equals() y hashCode() para usar correctamente en colecciones. Se añadió un método clone() para facilitar la exploración de estados. Se probó con estados conocidos (fáciles y difíciles) para validar la generación de movimientos.

N°	P02
Objetivo del prompt	Implementar los algoritmos de búsqueda no informados (BFS y DFS) para el 8-puzzle, con manejo de estados visitados, control de profundidad y generación de estadísticas de rendimiento.

Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Continúa la implementación del 8-puzzle en Java. Ahora necesito implementar algoritmos de búsqueda, considera lo siguiente: 1. BFS (Breadth-First Search) usando una cola FIFO (LinkedList). 2. DFS (Depth-First Search) usando una pila LIFO (Stack) con límite de profundidad de 30 para evitar bucles infinitos. 3. Ambos algoritmos deben: - Usar un HashSet para estados visitados. - Mantener el nodo padre para reconstruir la solución. - Retornar: nodos expandidos, costo de la solución, tiempo de ejecución y camino de movimientos. 4. Una clase principal que permita seleccionar el algoritmo y ejecutarlo con el estado inicial definido previamente. Incluye comentarios explicativos y manejo de casos donde no se encuentra solución."
Resultado / cómo se usó	Se obtuvieron las clases BFS y DFS con toda la lógica de búsqueda. Ambas implementaciones generaban estadísticas detalladas (nodos expandidos, profundidad, tiempo). Se añadió la reconstrucción del camino de solución a partir de los nodos padre.
Validación / ajuste del equipo	Se probaron ambos algoritmos con estados de complejidad variable. Se detectó que DFS sin límite de profundidad podía circular, por lo que se reforzó el límite según el mejor caso teórico de movimientos (30 movimientos maximo) y se implementó un control de nodos visitados en la misma ruta. Se ajustó la estructura de Nodo para incluir el operador que generó el estado. Se validó que BFS siempre encuentra la solución óptima (menor número de movimientos).

N°	P03
Objetivo del prompt	Implementar la estructura base de los algoritmos informados (Greedy Best-First y A*) con sus respectivas colas de prioridad y el sistema de evaluación heurística.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Para el 8-puzzle en Java, necesito implementar la base de los algoritmos informados, toma en cuenta lo siguiente antes de implementar: 1. Una interfaz o clase abstracta Buscador que defina el método buscar(Estado inicial). 2. La clase Nodo que contenga: estado, nodo padre, operador, profundidad (g), costo acumulado, y heurística (h). 3. Greedy Best-First Search con PriorityQueue ordenada

	por $h(n)$ (distancia Manhattan). 4. A con PriorityQueue ordenada por $f(n) = g(n) + h(n)$. 5. Mecanismo para evitar estados repetidos (HashSet de estados visitados). 6. Método para reconstruir el camino solución desde el nodo meta..**
Resultado / cómo se usó	Se obtuvieron las estructuras base de Nodo, Buscador, Greedy y A*. La implementación de PriorityQueue con Comparador permitió ordenar eficientemente los nodos según la heurística. El manejo de estados visitados evitó ciclos y mejoró el rendimiento.
Validación / ajuste del equipo	Se verificó que los comparadores funcionaran correctamente en PriorityQueue. Se ajustó la clase Nodo para implementar CompareTo y facilitar la ordenación en A*. Se probó que ambos algoritmos encontrarán solución para estados de complejidad media.

N°	P04
Objetivo del prompt	Completar la implementación de Greedy y A*, añadir heurísticas alternativas (fichas mal ubicadas), y crear un sistema de selección de heurística.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Completa la implementación de los algoritmos informados para el 8-puzzle: 1. Finalizar Greedy Best-First Search con la lógica completa de expansión de nodos. 2. Finalizar A con la lógica completa incluyendo la actualización de $g(n)$ cuando se encuentra un camino mejor. 3. Implementar heurísticas adicionales: - Fichas mal ubicadas (contar cuántas fichas no están en su posición objetivo). 4. Permitir al usuario seleccionar la heurística a usar en tiempo de ejecución. Incluye la justificación de por qué Manhattan es admisible y consistente.**
Resultado / cómo se usó	Se completaron los algoritmos Greedy y A* con todas las funcionalidades. Se completó la implementación de las 2 heurísticas diferentes (Manhattan, fichas mal ubicadas). El sistema permitía seleccionar la heurística en tiempo de ejecución para comparar su impacto en el rendimiento.
Validación / ajuste del equipo	Se verificó que la heurística fichas mal ubicadas fuera admisible. Se probó que A* con diferentes heurísticas encontraba soluciones óptimas pero con diferente

	eficiencia.
--	-------------

N°	P05
Objetivo del prompt	Crear un sistema completo de comparación que ejecute todos los algoritmos (BFS, DFS, Greedy, A* con múltiples estados de prueba y genera estadísticas detalladas.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Ahora necesito un sistema de comparación completo para todos los algoritmos de búsqueda del 8-puzzle: 1. Clase Comparativa que: - Ejecute BFS, DFS, Greedy y A* con los mismos estados de prueba. - Mida nodos expandidos, costo de solución, tiempo de ejecución (ms). - Determine completitud y optimalidad de cada algoritmo. 2. Múltiples casos de prueba: - Fácil (2-3 movimientos). - Medio (10-15 movimientos). - Difícil (25-31 movimientos). 3. Generación de tabla comparativa en consola con formato. - Pasos realizados. - Nodos expandidos. - Nodos generados - Tiempo de ejecución (ms)"*
Resultado / cómo se usó	Se obtuvo una clase ResultadoBusqueda que guardaba los parámetros pedidos de todos los algoritmos y generaba tablas formateadas en la consola.
Validación / ajuste del equipo	Se probó con 10 estados diferentes de cada categoría (fácil, medio, difícil). Se verificó que BFS y A* siempre encontraban soluciones óptimas. Se ajustó el límite de tiempo para evitar ejecuciones infinitas en DFS.

N°	P06
Objetivo del prompt	Crear una interfaz de usuario interactiva completa en consola, sistema de configuración de casos de prueba, y generar la documentación final del proyecto.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Para finalizar el proyecto del 8-puzzle modifica la interfaz de la consola en la clase main:

	1. Crear una interfaz de usuario interactiva en consola que permita: - Ingresar estado inicial manualmente. - Seleccionar entre estados predefinidos (fácil, medio, difícil). - Elegir algoritmo(s) a ejecutar (individual o todos). - Ver solución paso a paso con animación del tablero. 2. Añadir validación de entrada (números 0-8, formato correcto). 3. Incluir el despliegue de las métricas calculadas en la consola. La interfaz debe ser intuitiva y guiar al usuario paso a paso."
Resultado / cómo se usó	Se obtuvo un programa completo con interfaz interactiva que permitía todas las funcionalidades solicitadas. La visualización paso a paso de la solución era clara y didáctica.
Validación / ajuste del equipo	Se probó la interfaz con usuarios externos para validar su usabilidad. Se ajustaron los mensajes de error y la validación de entrada. Se ajustó la clase main para incluir los casos de prueba y se incluye una opción que ejecutaba automáticamente todos los casos de prueba.

Informe 2

N°	P01
Objetivo del prompt	Definir el modelo formal del Tic-Tac-Toe y la estructura base del código en Java, incluyendo representación del tablero, validación de movimientos, verificación de estado terminal y función de utilidad.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Necesito implementar el juego Tic-Tac-Toe (3 en raya) en Java proporciona el código para 1. La definición formal del juego (estados, jugadores, acciones legales, función terminal, utilidad). 2. Una clase Tablero que: - Represente el tablero 3x3 con caracteres ('X', 'O', ' '). - Valide movimientos (celda vacía). - Verifique si hay ganador (líneas: horizontales, verticales, diagonales). - Verifique si hay empate. - Devuelva utilidad (+10 para X, -10 para O, 0 para empate). 3. Una función de evaluación heurística que calcule líneas potenciales para cada jugador. 4. Un método main que permita jugar en consola (humano

	vs humano) para probar la lógica."
Resultado / cómo se usó	Se obtuvo una clase Board completa con toda la lógica del juego. La función de utilidad y evaluación heurística se implementaron para ser usadas posteriormente por Minimax. El modo humano vs humano permitió validar la corrección de las reglas del juego.
Validación / ajuste del equipo	Se verificaron todas las condiciones de victoria (8 líneas posibles). Se ajustó la evaluación heurística para que ponderara correctamente las líneas abiertas (sin oponente) y líneas bloqueadas. Se añadió un método getMovimientosLegales() que devuelve una lista de posiciones vacías. Se probó con múltiples partidas manuales para asegurar la detección temprana de ganadores.

N°	P02
Objetivo del prompt	Implementar el algoritmo Minimax en su versión completa (sin poda) para Tic-Tac-Toe, con conteo de nodos evaluados y retorno de la mejor jugada.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Para el Tic-Tac-Toe en Java, necesito implementar el algoritmo Minimax: 1. Clase Minimax que: - Tenga un método findBestMove(Board tablero, char jugador) que retorne la mejor posición (fila, columna). - Tenga un método recursivo minimax(Tablero tablero, int profundidad, char jugador). - Use la función de utilidad del tablero (+10 para MAX, -10 para MIN, 0 para empate). - Cuente el número total de nodos evaluados durante la búsqueda. 2. MAX es 'X', MIN es 'O'. 3. La evaluación debe considerar que X siempre empieza. 4. Incluir profundidad en el cálculo de utilidad para priorizar victorias más rápidas. 5. Método para mostrar estadísticas: nodos evaluados, profundidad alcanzada. El algoritmo debe ser completamente recursivo y explorar todo el árbol de juego hasta terminal."
Resultado / cómo se usó	Se obtuvo la clase Minimax con todos los métodos necesarios. La implementación recursiva evaluaba todo el árbol de juego y retornaba la mejor jugada. El conteo de nodos permitió medir el costo computacional de la búsqueda exhaustiva.

Validación / ajuste del equipo	Se verificó que el algoritmo siempre devolviera la jugada óptima en cada posición. Se ajustó la función de utilidad para penalizar victorias tardías (restando la profundidad). Se probó con todas las posiciones posibles del juego para validar el comportamiento. Se añadió un límite de profundidad para casos donde el juego podría ser infinito.
--------------------------------	--

N°	P03
Objetivo del prompt	Implementar Minimax con poda Alfa-Beta, con estadísticas detalladas de nodos evaluados y podados, y comparación con la versión sin poda.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	“Ahora implementa la poda Alfa-Beta para la clase MinMax para Tic-Tac-Toe, considera: 1. Clase MinimaxPoda que extienda o replique la estructura de Minimax. 2. Método mejorJugadaConPoda(Tablero tablero, char jugador). 3. Método recursivo minimaxPoda(Tablero tablero, int profundidad, char jugador, int alfa, int beta). 4. Contar: - Nodos evaluados totales. - Nodos podados (cuando se corta la rama). - Profundidad máxima alcanzada. 5. Método para comparar con Minimax sin poda desde la misma posición. 6. Mostrar: - Nodos sin poda vs con poda. - Porcentaje de reducción. - Tiempo de ejecución de cada uno. Incluir comentarios explicando el funcionamiento de la poda y por qué es correcta.”
Resultado / cómo se usó	Se obtuvo la clase MinimaxPoda completa. La implementación incluía el mantenimiento de alfa y beta durante la recursión, y el conteo preciso de nodos podados. El método de comparación permitía ver la mejora de rendimiento en tiempo real.
Validación / ajuste del equipo	Se verificó que ambos algoritmos devolvieran la misma jugada para todas las posiciones de prueba. Se validó el conteo de nodos podados comparando con la teoría. Se ajustó el orden de exploración de movimientos para maximizar la poda. Se probó en todas las posiciones del juego para asegurar la consistencia.

N°	P04
Objetivo del prompt	Implementar las clases de jugadores (humano y máquina) y optimizar el ordenamiento de movimientos para mejorar la eficiencia de la poda Alfa-Beta.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Para el Tic-Tac-Toe, necesito que apliques algunos cambios y estructuras lo siguiente: 1. Interfaz Jugador con método obtenerMovimiento(Tablero tablero). 2. Clase JugadorHumano que: - Solicite entrada por teclado (fila y columna). - Valide que la entrada sea correcta (0-2) y celda vacía. 3. Clase JugadorMaquina que: - Use Minimax con poda Alfa-Beta. - Permita configurar el algoritmo (con/sin poda). 4. Ordenamiento de movimientos (opcional): - Evaluar movimientos con heurística simple. - Ordenar movimientos para mejorar eficiencia de poda. 5. Método para visualizar el tablero después de cada movimiento. El ordenamiento de movimientos debe probarse y registrar su impacto en la poda."
Resultado / cómo se usó	Se obtuvieron clases de jugadores completamente funcionales. El ordenamiento de movimientos priorizaba el centro, luego esquinas, luego bordes, mejorando significativamente la eficiencia de la poda. La visualización del tablero después de cada movimiento mejoraba la experiencia de juego.
Validación / ajuste del equipo	Se probó el ordenamiento de movimientos en múltiples posiciones y se midió el impacto en la poda. Se ajustó la interfaz de JugadorHumano para manejar entradas inválidas de manera amigable.

N°	P05
Objetivo del prompt	Implementar los diferentes modos de juego (Humano vs Máquina, Máquina vs Máquina) con selección de algoritmo y configuración de profundidad.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Crea una clase para gestionar las partidas de Tic-Tac-Toe que incuya: 1. Clase Juego que gestione: - El flujo del juego (turnos, validación de movimientos). - Verificación de estado terminal después de cada

	movimiento. - Anuncio del ganador o empate. 2. Modos de juego: - Humano (O) vs Máquina (X) - el humano elige su ficha. - Máquina (X) vs Máquina (O) - con selección de algoritmo para cada una. 3. Configuración de profundidad máxima para la máquina (fácil=1, medio=3, difícil=5, experto=9). 4. Menú principal que permita: - Seleccionar modo de juego. - Configurar profundidad. - Elegir algoritmo con/sin poda. - Iniciar partida. 5. Estadísticas acumuladas de la partida. El juego debe ser interactivo y fácil de usar."
Resultado / cómo se usó	Se obtuvo una clase Juego completa que gestiona todos los modos y configuraciones. El menú principal era claro y permitía navegar fácilmente entre opciones. Las estadísticas acumuladas mostraban el rendimiento de los algoritmos durante la partida.
Validación / ajuste del equipo	Se probaron todos los modos de juego con diferentes configuraciones de profundidad. Se verificó que la máquina nunca pierde (siempre empata o gana) cuando se usa profundidad total. Se detectó un error en el que la máquina siempre escoge el centro al inicio de la partida.

N°	P06
Objetivo del prompt	Realizar un análisis comparativo exhaustivo ejecutando múltiples partidas, generar tablas y gráficos de rendimiento, añadir la funcionalidad de decidir quien abre el juego y corregir que el primer movimiento de la máquina sea siempre el centro
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Para finalizar el proyecto de Tic-Tac-Toe, realiza algunos ajustes y añade lo siguiente: 1. Clase AnalisisRendimiento que: - Ejecute 100 partidas máquina vs máquina (50 con X, 50 con O). - Compare Minimax sin poda vs con poda. - Mida: nodos promedio, tiempo promedio, profundidad promedio. 2. Tablas comparativas para profundidades 3, 5, 7 y 9. 3. Registro de: - Nodos evaluados vs profundidad. - Tiempo de ejecución vs profundidad. - Porcentaje de reducción con poda.

	4. Añade la opción de decidir quien empieza la partida en el modo Humano (O) vs Maquina (X). 5. En el modo Humano (O) vs Maquina (X), la maquina siempre empieza por el centro, debido a que en el método de <code>findeBestMoves</code> de la clase <code>MinMax</code> tiene como prioridad principal el centro evitando movimientos en la esquinas, ajusta el algoritmo para que el primer movimiento pueda variar de la posición (1,1)
Resultado / cómo se usó	Se obtuvo un análisis completo con 100 partidas por configuración. Los datos generados permitieron crear tablas y gráficos detallados. Se añadieron las funciones requeridas y se corrigieron las clases mencionadas
Validación / ajuste del equipo	Se ejecutó el análisis en diferentes máquinas para validar la consistencia de los resultados. Se verificó que los porcentajes de reducción de nodos estuvieran dentro de los rangos esperados (50-75% aproximadamente).

Informe 3

Prompt P-01 — Análisis exploratorio (EDA) y preprocesamiento

Campo	Contenido
N°	P-01
Objetivo del prompt	Realizar el análisis exploratorio completo del dataset de VidaCare, incluyendo estadísticos descriptivos, detección de valores nulos, matriz de correlaciones, scatterplots de las variables más correlacionadas con el target y preparar el pipeline de preprocesamiento (imputación de nulos, codificación one-hot de categóricas, escalado con <code>StandardScaler</code> y división train/test 80/20).
Herramienta / modelo de IA	opencode (DeepSeek-V4-Flash-Free)

Texto del prompt	Carga el archivo pacientes_vidacare.csv en un DataFrame de pandas. Realiza un EDA completo: (1) muestra dimensiones, primeras filas e información general; (2) identifica valores nulos por columna y su porcentaje; (3) genera estadísticas descriptivas de las variables numéricas; (4) grafica la distribución de gasto_total_anual (histograma + boxplot) e imprime media, mediana, desviación estándar, mínimo y máximo; (5) genera una matriz de correlaciones con heatmap para todas las variables numéricas y muestra la correlación con gasto_total_anual ordenada de mayor a menor; (6) crea scatterplots de las 3 variables más correlacionadas vs el target. Luego preprocesa los datos con ColumnTransformer, SimpleImputer, OneHotEncoder, StandardScaler y división train/test 80/20.
Resultado / cómo se usó	Se generaron 7 bloques de código con información del DataFrame, análisis de nulos (~4%), estadísticas descriptivas, gráficos del target, heatmap de correlaciones, scatterplots y pipeline de preprocesamiento listo para el modelado.
Validación / ajuste del equipo	Se verificó que las correlaciones fueran coherentes, que OneHotEncoder generara tres variables adicionales y que StandardScaler centrara correctamente los datos.

Prompt P-02 — Regresión Lineal Simple y Múltiple (OLS)

Campo	Contenido
N°	P-02
Objetivo del prompt	Implementar modelos de Regresión Lineal Simple y Múltiple, calcular MSE, RMSE, MAE y R^2 y visualizar resultados.
Herramienta / modelo de IA	opencode (DeepSeek-V4-Flash-Free)
Texto del prompt	Implementa Regresión Lineal Simple utilizando la variable más correlacionada con el target (frecuencia_citas) y Regresión Lineal Múltiple con todas las variables. Calcula MSE, RMSE, MAE y R^2 . Genera gráficos de valores reales vs predichos, residuos y muestra los coeficientes ordenados por magnitud.
Resultado / cómo se usó	El modelo Simple obtuvo aproximadamente $R^2=0.67$ y $RMSE\approx 590$. El modelo Múltiple alcanzó $R^2\approx 0.89$ y $RMSE\approx 340$, mostrando una mejora importante.
Validación / ajuste del equipo	Se compararon ambos modelos y se verificó la coherencia de los coeficientes y del RMSE respecto a la escala del problema.

Prompt P-03 — Regresión Polinómica

Campo	Contenido
N°	P-03
Objetivo del prompt	Implementar Regresión Polinómica comparando los grados 1, 2 y 3 para encontrar el mejor equilibrio entre precisión y complejidad.
Herramienta / modelo de IA	opencode (DeepSeek-V4-Flash-Free)
Texto del prompt	Crear un pipeline con StandardScaler, PolynomialFeatures y LinearRegression. Comparar los grados 1, 2 y 3 mostrando RMSE, R^2 y número de características generadas.
Resultado / cómo se usó	El grado 2 generó 55 variables y obtuvo $R^2 \approx 0.90$. El grado 3 produjo 220 variables con riesgo de sobreajuste.
Validación / ajuste del equipo	Se comprobó que el grado 2 ofrecía el mejor equilibrio entre rendimiento y complejidad.

Prompt P-04 — Ridge y Lasso

Campo	Contenido
N°	P-04
Objetivo del prompt	Implementar Ridge y Lasso utilizando validación cruzada para seleccionar el mejor alpha y analizar el efecto de la regularización.
Herramienta / modelo de IA	opencode (DeepSeek-V4-Flash-Free)
Texto del prompt	Implementar RidgeCV y LassoCV probando diferentes valores de alpha. Mostrar métricas de desempeño, mejor alpha y los coeficientes resultantes, identificando las variables eliminadas por Lasso.
Resultado / cómo se usó	Ridge obtuvo un desempeño similar al OLS Múltiple. Lasso eliminó variables poco relevantes y realizó selección automática de características.
Validación / ajuste del equipo	Se verificó la estabilidad de los alphas y la coherencia entre las variables eliminadas y el análisis de correlaciones.

Prompt P-05 — Gradient Boosting

Campo	Contenido
N°	P-05
Objetivo del prompt	Implementar Gradient Boosting, evaluar su desempeño e identificar las características más importantes.
Herramienta / modelo de IA	opencode (DeepSeek-V4-Flash-Free)
Texto del prompt	Entrenar un GradientBoostingRegressor, calcular métricas de desempeño, mostrar la importancia de variables y realizar validación cruzada de cinco particiones.
Resultado / cómo se usó	Fue el mejor modelo con $R^2 \approx 0.92$ y $RMSE \approx 290$. Las variables más importantes fueron frecuencia_citas y gasto_promedio_cita.
Validación / ajuste del equipo	Se confirmó que no existía sobreajuste y que las importancias coincidían con el análisis exploratorio.

Prompt P-06 — Comparación de modelos y conclusiones

Campo	Contenido
N°	P-06
Objetivo del prompt	Comparar todos los modelos, seleccionar el mejor y generar conclusiones de negocio.
Herramienta / modelo de IA	opencode (DeepSeek-V4-Flash-Free)
Texto del prompt	Consolidar todos los modelos en una tabla comparativa, identificar el mejor según RMSE y R^2 , generar gráficos comparativos y redactar conclusiones y recomendaciones para el equipo de marketing de VidaCare.
Resultado / cómo se usó	Gradient Boosting fue el mejor modelo ($RMSE \approx 290$ y $R^2 \approx 0.92$). Se concluyó que frecuencia_citas y gasto_promedio_cita son los factores más influyentes.
Validación / ajuste del equipo	Se comprobó que todos los modelos estuvieran incluidos en la comparación y que las recomendaciones fueran específicas para el contexto del proyecto.

Informe 4

Campo	Contenido
N°	P-01
Objetivo del prompt	Seleccionar los modelos de clasificación adecuados para el problema de abandono de pacientes.
Herramienta / modelo de IA	ChatGPT (GPT-5.5)
Texto del prompt	¿Qué modelos de clasificación supervisada son adecuados para predecir una variable binaria como abandono en un dataset de pacientes y cuáles recomienda scikit-learn para compararlos?
Resultado / cómo se usó	Se obtuvo una recomendación de implementar Regresión Logística, KNN, SVM, Random Forest, Gradient Boosting y un Voting Classifier como modelos de comparación.
Validación / ajuste del equipo	El equipo contrastó la propuesta con la documentación de scikit-learn y seleccionó los modelos que mejor se adaptaban al objetivo del proyecto.

Campo	Contenido
N°	P-02
Objetivo del prompt	Definir un flujo correcto de preprocesamiento para un dataset con variables numéricas y categóricas.
Herramienta / modelo de IA	ChatGPT (GPT-5.5)
Texto del prompt	Mi dataset contiene variables numéricas, categóricas y algunos valores nulos. ¿Cuál sería un flujo adecuado de preprocesamiento utilizando Pipeline y ColumnTransformer en scikit-learn?
Resultado / cómo se usó	Se obtuvo una propuesta de imputación de datos, escalamiento con StandardScaler, codificación One-Hot Encoding e integración mediante Pipeline y ColumnTransformer.
Validación / ajuste del equipo	El flujo fue adaptado a las características del dataset VidaCare Perú y posteriormente implementado y probado en el notebook.

Campo	Contenido
N°	P-03
Objetivo del prompt	Determinar qué hiperparámetros optimizar en cada algoritmo de clasificación.
Herramienta / modelo de IA	ChatGPT (GPT-5.5)
Texto del prompt	¿Qué hiperparámetros conviene optimizar mediante GridSearchCV para Regresión Logística, KNN, SVM, Random Forest y Gradient Boosting?
Resultado / cómo se usó	Se identificaron los principales hiperparámetros de cada algoritmo y se definieron las combinaciones evaluadas mediante GridSearchCV.
Validación / ajuste del equipo	Los rangos sugeridos fueron ajustados según el tiempo de ejecución disponible y el comportamiento observado durante las pruebas.

Campo	Contenido
N°	P-04
Objetivo del prompt	Identificar las métricas más apropiadas para comparar modelos de clasificación.
Herramienta / modelo de IA	ChatGPT (GPT-5.5)
Texto del prompt	¿Qué métricas son las más adecuadas para evaluar modelos de clasificación binaria orientados a predecir abandono de clientes y cómo deben interpretarse?
Resultado / cómo se usó	Se seleccionaron Accuracy, Precision, Recall, F1-Score, ROC-AUC y la matriz de confusión como métricas principales para la evaluación.
Validación / ajuste del equipo	El equipo verificó cada métrica utilizando las funciones de evaluación de scikit-learn y decidió utilizar el F1-Score como criterio principal de selección del modelo.

Campo	Contenido
--------------	------------------

N°	P-05
Objetivo del prompt	Interpretar los resultados obtenidos durante el análisis exploratorio de datos.
Herramienta / modelo de IA	ChatGPT (GPT-5.5)
Texto del prompt	¿Cómo interpretar un countplot, una matriz de correlación y diagramas de caja en un proyecto de clasificación de abandono de pacientes?
Resultado / cómo se usó	Se obtuvieron orientaciones para explicar la distribución de la variable objetivo, las relaciones entre variables y las diferencias observadas entre pacientes que permanecen y abandonan.
Validación / ajuste del equipo	Las interpretaciones fueron contrastadas con los gráficos generados en el notebook y adaptadas al contexto de VidaCare Perú.

Campo	Contenido
N°	P-06
Objetivo del prompt	Justificar la selección del modelo final y explicar su utilidad para el negocio.
Herramienta / modelo de IA	ChatGPT (GPT-5.5)
Texto del prompt	¿Cómo justificar la selección del modelo con mejor F1-Score y relacionarlo con el problema de negocio de una empresa de salud digital?
Resultado / cómo se usó	Se obtuvo una guía para argumentar la elección del modelo final considerando el equilibrio entre Precision y Recall y su impacto en la identificación de pacientes con riesgo de abandono.
Validación / ajuste del equipo	El equipo validó la justificación con las métricas obtenidas en la experimentación y la adaptó a los resultados reales del modelo KNN.

Informe 5

Prompt P-01 — Generacion del dataset sintetico de VidaCare Peru

Campo	Contenido
N°	P-01
Objetivo del prompt	Generar el dataset sintetico de VidaCare Peru
Herramienta / modelo de IA	opencode (DeepSeek-V4)
Texto del prompt	"Genera codigo Python con numpy, pandas y random para crear un dataset sintetico de 1500 pacientes de una empresa peruana de salud digital llamada 'VidaCare Peru'. El dataset debe tener 12 variables: id_paciente (P0001-P1500), edad (18-85 distribucion normal con media 45), ingreso_mensual (1000-8000 soles, correlacionado positivamente con edad y con ruido gaussiano), frecuencia_citas (0-36, distribucion Poisson donde Basico=4, Preventivo=8, Premium=14, mas un bonus segun ingreso), gasto_promedio_cita (30-350 soles, dependiente del tipo de plan e ingreso), antiguedad_meses (1-60 uniforme), recencia_dias (1-200, inversamente correlacionada con antiguedad y frecuencia), tipo_plan (categorica: Basico 50%, Preventivo 30%, Premium 20%), canal_atencion (Presencial 55%, Telemedicina 45%), gasto_total_anual (calculado como frecuencia x gasto_promedio x 0.95 mas ruido gaussiano), abandono (variable objetivo binaria 0/1 dependiente de baja frecuencia y alta recencia mediante funcion sigmoide), num_condiciones_cronicas (0-5, relacionado con la edad). Introduce correlaciones realistas entre las variables, anade 5% de ruido aleatorio en frecuencia y gasto, y un 4% de valores nulos en columnas seleccionadas. Fija la semilla aleatoria en 42 para garantizar reproducibilidad. Exporta el DataFrame resultante a un archivo CSV. El script debe producir un dataset que tenga estructura de clustering visible (4 segmentos naturales) para que funcione bien con K-means."
Resultado / cómo se usó	Se obtuvo el script generador completo con todas las variables, correlaciones, ruido y nulos. El equipo lo ejecuto y produjo el archivo pacientes_vidacare.csv con 1500 filas y 12 columnas. Se revisaron las distribuciones estadisticas y se verifico que las correlaciones fueran coherentes (p. ej., frecuencia_citas con tipo_plan, recencia_dias inversa con antiguedad). Se genero tambien el notebook generar_dataset_vidacare.ipynb que documenta paso a paso la generacion.

Validación / ajuste del equipo	Se verifico la coherencia de las correlaciones con un analisis exploratorio (matriz de correlacion, boxplots). Se corrigio el rango de frecuencia_citas para evitar valores extremos y se ajustaron los parametros de Poisson para que los segmentos fueran distinguibles. Se confirmo que la tasa de abandono rondara el 25-30% para que la variable objetivo estuviera balanceada.
---------------------------------------	--

Prompt P-02 — K-means y barrido de K con metricas internas

Campo	Contenido
N°	P-02
Objetivo del prompt	Implementar el barrido de K-means con las 4 metricas internas para determinar el numero optimo de clusters.
Herramienta / modelo de IA	opencode (DeepSeek-V4)
Texto del prompt	"Genera codigo Python utilizando scikit-learn para ejecutar K-means sobre un conjunto de datos estandarizados (StandardScaler) probando valores de k desde 2 hasta 10. Para cada valor de k, calcula e imprime las siguientes 4 metricas de evaluacion interna: inercia (WCSS) usando el atributo inertia_ del modelo KMeans, coeficiente de silueta promedio con silhouette_score, indice de Davies-Bouldin con davies_bouldin_score, e indice de Calinski-Harabasz con calinski_harabasz_score. Almacena los resultados en un DataFrame y muestra la tabla completa. Luego: (a) genera un grafico del metodo del codo (inercia vs k) con linea roja punteada senalando el posible codo, (b) genera un grafico de silueta promedio vs k marcando el valor maximo, (c) genera un grafico de Davies-Bouldin vs k marcando el valor minimo, y (d) genera un grafico de Calinski-Harabasz vs k marcando el valor maximo. Organiza los 4 graficos en una figura de 2x2. Finalmente, determina el k optimo por triangulacion: el valor de k que aparezca como optimo en al menos 2 de las 3 metricas (excluyendo la inercia que solo sirve como referencia del codo). Usa random_state=42 y n_init=10 para reproducibilidad."
Resultado / cómo se usó	Se implemento el barrido completo de K-means generando la tabla de resultados y los 4 graficos de validacion. Se determino que k=4 era el optimo por triangulacion: el metodo del codo senalo k=4 como punto de inflexion, y las metricas de Davies-Bouldin y Calinski-Harabasz tambien favorecieron k=4. La silueta fue marginalmente mejor en k=5, pero por consenso se eligio k=4 por ser mas interpretable para el negocio.

Validación / ajuste del equipo	Se amplio el rango de k de 2-8 a 2-10 para tener una vision mas completa. Se verifico que los valores de las metricas fueran consistentes con ejecuciones repetidas. Se confirmo visualmente con el grafico de silueta por punto que ningun cluster presentaba valores negativos significativos.
---------------------------------------	--

Prompt P-03 — DBSCAN con exclusion de ruido

Campo	Contenido
N°	P-03
Objetivo del prompt	Implementar DBSCAN con evaluacion de metricas excluyendo correctamente los puntos de ruido.
Herramienta / modelo de IA	opencode (DeepSeek-V4)
Texto del prompt	"Genera codigo Python para evaluar el algoritmo DBSCAN sobre datos estandarizados. Crea una funcion llamada evaluar_dbscan que reciba la matriz X, el valor de eps y min_samples (default 5). Dentro de la funcion: ejecuta DBSCAN con esos parametros, identifica los puntos de ruido (etiqueta -1), cuenta el numero de clusters validos (excluyendo el ruido) y calcula el porcentaje de ruido. Si hay al menos 2 clusters validos, filtra la matriz X para quedarse solo con los puntos que NO son ruido (mascara booleana donde etiquetas != -1) y calcula silhouette_score, davies_bouldin_score y calinski_harabasz_score exclusivamente sobre esos puntos. Si hay menos de 2 clusters, muestra un mensaje indicando que las metricas no estan definidas. Prueba la funcion con eps = [0.3, 0.5, 0.7, 0.9, 1.2] y muestra una tabla con los resultados: eps, numero de clusters, porcentaje de ruido, silueta, Davies-Bouldin y Calinski-Harabasz. Es importante NO incluir los puntos de ruido en el calculo de las metricas porque la silueta requiere al menos 2 clusters y el ruido distorsiona las medidas de compacidad y separacion."
Resultado / cómo se usó	Se creo la funcion evaluar_dbscan que maneja correctamente la exclusion de ruido. Los resultados mostraron que con eps=0.7 se obtienen 4 clusters con solo 3% de ruido y metricas superiores a K-means. Esto confirmo de forma independiente la estructura de 4 segmentos sugerida por el metodo del codo. DBSCAN con eps=0.5 produjo 4 clusters pero con mas ruido (12%), mientras que eps=0.9 fusiono algunos clusters.
Validación / ajuste del equipo	Se verifico que al incluir los puntos de ruido en el calculo de metricas, los valores empeoraban significativamente, confirmando la necesidad de excluirlos. Se comprobe que con menos de 2 clusters validos la

	funcion retorna correctamente sin calcular metricas. Se ajusto el rango de eps para cubrir desde valores muy estrictos (0.3) hasta muy laxos (1.2).
--	---

Prompt P-04 — Interpretacion de perfiles de segmentos

Campo	Contenido
N°	P-05
Objetivo del prompt	Interpretar los perfiles promedio de los segmentos obtenidos con K-means y asignar nombres de negocio con estrategias comerciales.
Herramienta / modelo de IA	opencode (DeepSeek-V4)
Texto del prompt	"Tengo un DataFrame con los resultados de un clustering K-means (k=4) aplicado a pacientes de una empresa peruana de salud digital llamada VidaCare Peru. Las variables de clustering fueron: frecuencia_citas (numero de citas al ano), gasto_promedio_cita (gasto promedio por cita en soles), antiguedad_meses (meses desde el registro) y recencia_dias (dias desde la ultima cita). Los perfiles promedio de cada cluster son los siguientes: Cluster 0: frecuencia=28, gasto=245, antiguedad=42, recencia=10; Cluster 1: frecuencia=14, gasto=155, antiguedad=28, recencia=32; Cluster 2: frecuencia=6, gasto=110, antiguedad=6, recencia=45; Cluster 3: frecuencia=4, gasto=80, antiguedad=35, recencia=98. Ademàs, tengo informacion sobre tipo de plan (Basico/ Preventivo/Premium), tasa de abandono por cluster y canal de atencion. Ayudame a: (1) asignar un nombre comercial a cada segmento que refleje su perfil en el contexto de salud digital peruano, (2) sugerir una estrategia de negocio especifica para cada segmento (marketing, retencion, fidelizacion), (3) redactar una interpretacion de 3-4 lineas por segmento explicando por que tienen ese comportamiento. Los nombres deben ser en espanol y pensados para una presentacion a la gerencia de marketing de VidaCare Peru."
Resultado cómo se usó /	Se obtuvieron los 4 nombres de segmento con su interpretacion de negocio: (1) Fidelizado Premium — pacientes con alta frecuencia y gasto, baja recencia, son el nucleo de ingresos; estrategia: plan VIP y referidos. (2) Regular Preventivo — valores moderados, base estable; estrategia: campanas de prevencion y chequeos. (3) Nuevo Exploratorio — baja antiguedad, explorando el servicio; estrategia: onboarding digital y ofertas de bienvenida. (4) Inactivo / En Riesgo — alta recencia, riesgo de abandono; estrategia: reactivacion con telemedicina gratuita.

Validación / ajuste del equipo	El equipo ajusto los nombres para que reflejaran mejor la terminología del sector salud peruano. Se verifico que las estrategias fueran factibles operativamente (ej. la telemedicina gratuita es viable en VidaCare). Se anadio la correspondencia con tipo de plan y canal para enriquecer los perfiles. Se reorganizaron los clusters por orden de frecuencia descendente para que los nombres fueran intuitivos.
---------------------------------------	--

Informe 6

Prompt P-01 — Interpretación de las cargas (loadings) de PCA

Campo	Contenido
N°	P-01
Objetivo del prompt	Interpretar en términos de negocio la tabla de cargas (loadings) obtenida de PCA, identificando qué variables dominan PC1 y PC2 y qué significado tienen para el perfil de un paciente de VidaCare.
Herramienta / modelo de IA	Claude (Anthropic)
Texto del prompt	"Aquí está la tabla de loadings de PC1 y PC2 para las 11 variables del dataset de pacientes. Ayúdame a interpretar que representa cada componente en términos del negocio de VidaCare, basándose únicamente en las variables con mayor carga absoluta en cada uno." Introduce correlaciones realistas entre las variables, añade 5% de ruido aleatorio en frecuencia y gasto, y un 4% de valores nulos en columnas seleccionadas. Fija la semilla aleatoria en 42 para garantizar la reproducibilidad. Exporta el Data Frame resultante a un archivo CSV. El script debe producir un dataset que tenga estructura de clustering visible (4 segmentos naturales) para que funcione bien con K-means."
Resultado / cómo se usó	La primera versión de la interpretación generada afirmaba que PC1 estaba dominado por "frecuencia, antigüedad, recencia y gasto", agrupando `antigüedad_meses` junto con las demás variables de comportamiento. Esta interpretación se uso como borrador inicial en la sección 4 del notebook.
Validación / ajuste del equipo	Al revisar la tabla de loadings real generada por el código (<code>loadings.round(3)</code>), el equipo detectó que `antigüedad_meses` tenía una carga de apenas 0.032 en PC1 y -0.076 en PC2 — es decir, prácticamente no aportaba a ningún componente, contradiciendo la interpretación inicial. Se corrigió el texto para reflejar los valores

	reales: PC1 quedo descrito solo con `gasto_total_anual`, `gasto_promedio_cita`, `frecuencia_citas` y `tipo_plan_Premium`, y se agrego una nota explícita señalando que `antiguedad_meses` no carga fuerte en ningun eje. Este ajuste evitó reportar una interpretacion que no correspondia con los numeros obtenidos.
--	---

Prompt P-02 — Diagnóstico de un resultado inesperado (ROC-AUC cercano al azar)

Campo	Contenido
N°	P-02
Objetivo del prompt	Entender por qué el modelo de clasificación de `abandono` obtiene un ROC-AUC cercano a 0.5 tanto con las variables originales como con los componentes de PCA, y determinar si esto refleja un error en el código o una limitación real de los datos.
Herramienta / modelo de IA	Claude (Anthropic)
Texto del prompt	"El ROC-AUC de mi modelo de clasificación de abandono es 0.46 con variables originales y 0.49 con PCA, casi igual al azar en ambos casos. ¿Esto indica un error en mi pipeline (encoding, escalamiento, fuga de datos) o puede ser un problema del dataset? Ayúdame a verificarlo antes de sacar conclusiones."
Resultado / cómo se usó	Se sugiere verificar la señal predictiva real calculando la correlación de `abandono` con las variables que, segun la documentacion del dataset (`contexto_empresa_vidacare.md`), deberian ser sus principales causas: `frecuencia_citas` y `recencia_dias`. Se encontraron correlaciones muy bajas (-0.08 y 0.06 respectivamente), y un modelo simple entrenado solo con esas dos variables tampoco supera un AUC de 0.54.
Validación / ajuste del equipo	El equipo confirmó que no había error de pipeline: el problema era que la fórmula generadora de `abandono` en el dataset sintético produce una señal débil por diseño (coeficientes pequeños en la función sigmoide). Esta verificación, ya documentada primero en el Informe 4, se referenció explícitamente en la sección 7 del Informe 6 para no repetir el análisis desde cero y para evitar una conclusión errónea de que PCA "fallo".

Prompt P-03 — Corrección de un error de código por cambio de API en scikit-learn

Campo	Contenido
N°	P-03

Objetivo del prompt	Corregir un error de ejecución (`TypeError`) al llamar a `TSNE(...)` con el parámetro `n_iter`, y entender la causa antes de aplicar el cambio.
Herramienta / modelo de IA	Claude (Anthropic)
Texto del prompt	"Al ejecutar TSNE con n_iter=1000 me sale un error. ¿Por qué ocurre y cuál es la forma correcta de fijar el número de iteraciones en la versión de scikit-learn que estoy usando?"
Resultado / cómo se usó	Se explicó que en versiones recientes de scikit-learn el parámetro `n_iter` de `TSNE` fue renombrado a `max_iter`, y que por eso la llamada fallaba con la versión instalada. Se corrigió la celda correspondiente reemplazando `n_iter=1000` por `max_iter=1000`.
Validación / ajuste del equipo	Se volvió a ejecutar el notebook completo (Restart & Run All) después del cambio para confirmar que ya no arrojaba errores, y se verificó que los resultados de t-SNE (coordenadas y gráfico) no se vieran afectados por la corrección, ya que el valor del parámetro fue el mismo, sólo cambió su nombre.